E-COMMERCE SALES SQL ANALYSIS

Business Questions • SQL Queries • Output Evidence • Business Conclusions

Database	MySQL / wallmart
Tables	customers + orders
Analysis Questions	16
Primary Focus	E-commerce sales & customer analytics

Prepared from the supplied MySQL analysis script and output screenshots

1. Executive Summary

The supplied dataset contains 5 customers and 5 orders across Electronics, Grocery and Fashion. Gross revenue is 10,450. Grocery is the largest revenue category, Card is the leading payment method, 25% of ordering customers are repeat customers, February shows 26.36% net-revenue MoM growth, and Fashion has the slowest average delivery time.

Customers **5**	**Orders** **5**	**Gross Revenue** **10,450**
AOV* **2,090**	**Repeat Customers*** **25%**	**Feb MoM Net Growth** **26.36%**

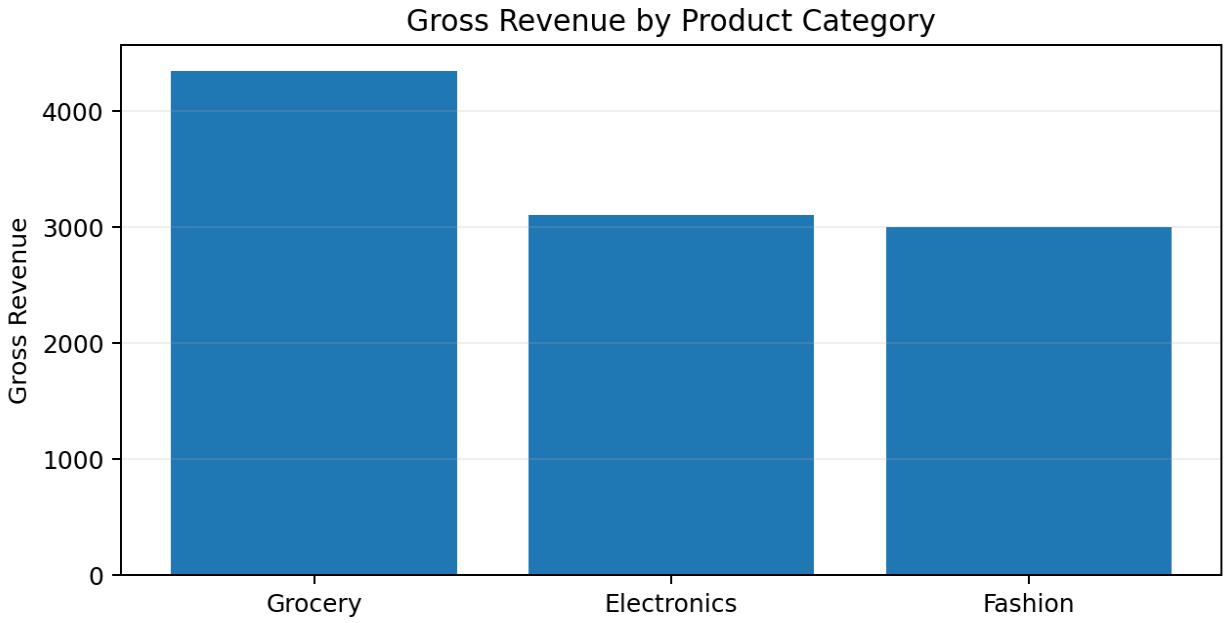


Figure 1. Gross revenue by category, derived from the supplied Q4 output.

2. Dataset & Data Model

The SQL script creates the wallmart database and two related tables: customers and orders. The orders table references customers.customer_id as a foreign key.

Table	Key	Important Fields	Role
customers	customer_id (PK)	name, city, state, signup_date, customer_type	Customer master
orders	order_id; customer_id (FK)	date, category, product, quantity, price, discount, payment, status, delivery	Transaction fact

SQL concepts: COUNT, SUM, AVG, GROUP BY, ORDER BY, LIMIT, HAVING, LEFT JOIN, INNER JOIN, CTEs, CASE, date formatting, LAG(), RANK(), aggregation, customer segmentation and MoM growth.

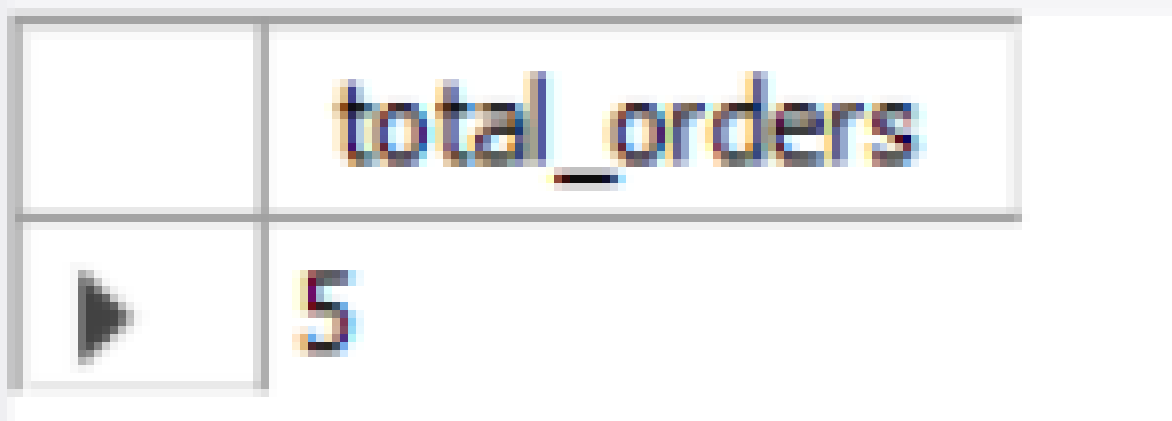
1. How many total orders has the company received?

QUESTION
How many total orders has the company received?

SQL Query

```
SELECT
COUNT(*) AS total_orders
FROM
customers;
```

Output



Output evidence supplied for Question 1.

Conclusion

BUSINESS INTERPRETATION
The supplied output shows 5, but the query counts rows in the customers table rather than the orders table. Because both tables currently contain 5 rows, the displayed number happens to equal the order count. For production reporting, the query should count order_id from orders.

Validation Note

CHECK BEFORE PRESENTATION
Q1: The question asks for total orders, but the query counts rows from customers. The displayed result is coincidentally 5 because both tables contain five rows.

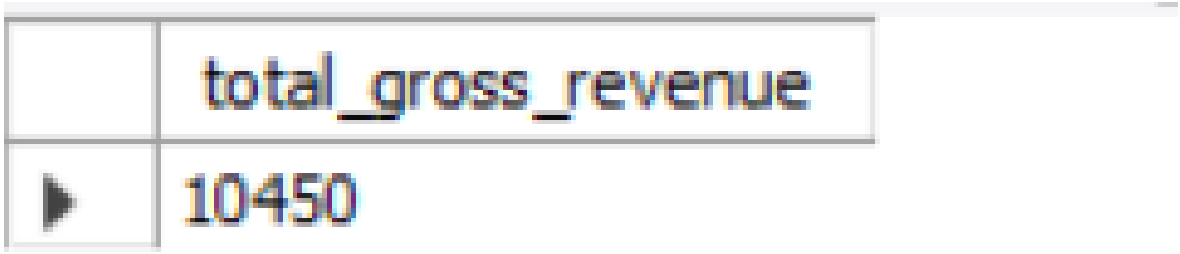
2. Calculate the gross revenue generated by the company.

QUESTION
Calculate the gross revenue generated by the company.

SQL Query

```
SELECT
SUM(quantity * unit_price) AS total_gross_revenue
FROM orders;
```

Output



	total_gross_revenue
▶	10450

Output evidence supplied for Question 2.

Conclusion

BUSINESS INTERPRETATION
The analysis reports gross revenue of 10,450 across the five orders. This is the pre-discount transaction value and provides the baseline for revenue analysis.

3. Calculate the company's Average Order Value (AOV).

QUESTION
Calculate the company's Average Order Value (AOV).

SQL Query

```
SELECT
AVG(quantity * unit_price) AS avg_order_value
FROM orders;
```

Output

	total_gross_revenue
▶	10450

Output evidence supplied for Question 3.

Conclusion

BUSINESS INTERPRETATION
The supplied screenshot repeats the gross-revenue output (10,450), while the supplied query uses AVG(quantity × unit_price). With five orders, the AOV implied by the query is 2,090. The screenshot therefore appears inconsistent with the query and should be refreshed before presentation.

Validation Note

CHECK BEFORE PRESENTATION
Q3: The screenshot repeats Q2's gross-revenue result, while the SQL calculates an average. Expected AOV from the supplied data/query is 2,090.

4. Find: Total orders, Total quantity sold, Total revenue for each product_category.

QUESTION

Find: Total orders, Total quantity sold, Total revenue for each product_category.

SQL Query

```
SELECT
product_category,
COUNT(order_id) AS total_orders,
SUM(quantity) AS total_quantity_sold,
SUM(quantity * unit_price) AS total_gross_revenue
FROM orders
GROUP BY product_category;
```

Output

product_category	total_orders	total_quantity_sold	total_gross_revenue
Electronics	2	3	3100
Grocery	2	8	4350
Fashion	1	1	3000

Output evidence supplied for Question 4.

Conclusion

BUSINESS INTERPRETATION

Grocery generates the highest gross revenue at 4,350 and also has the highest quantity sold at 8. Electronics contributes 3,100 and Fashion 3,000. Grocery is therefore the strongest category in the supplied sample.

5. Which payment method generates the highest revenue?

QUESTION

Which payment method generates the highest revenue?

SQL Query

```
SELECT
payment_method,
SUM(quantity * unit_price) AS total_gross_revenue
FROM
orders
GROUP BY payment_method
ORDER BY SUM(quantity * unit_price) DESC;
```

Output

payment_method	total_gross_revenue
Card	6000
UPI	3100
COD	1350

Output evidence supplied for Question 5.

Conclusion

BUSINESS INTERPRETATION

Card payments generate the highest gross revenue at 6,000, followed by UPI at 3,100 and COD at 1,350. Card transactions contribute the largest revenue share in this dataset.

6. Customer-wise Revenue - Join

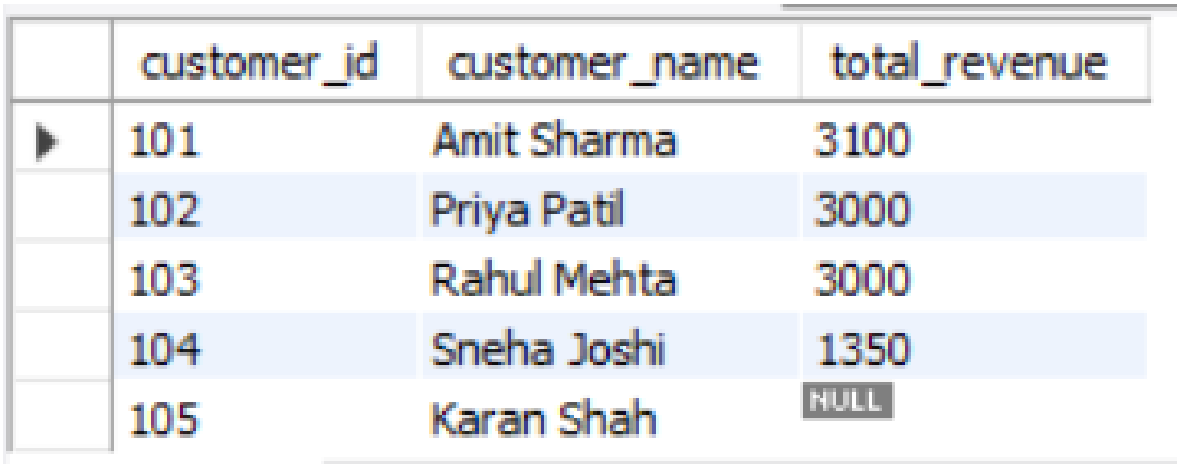
QUESTION

Customer-wise Revenue - Join

SQL Query

```
SELECT
c.customer_id,
c.customer_name,
SUM(o.quantity * o.unit_price) AS total_revenue
FROM
customers c
LEFT JOIN
orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id ;
```

Output



customer_id	customer_name	total_revenue
101	Amit Sharma	3100
102	Priya Patil	3000
103	Rahul Mehta	3000
104	Sneha Joshi	1350
105	Karan Shah	NULL

Output evidence supplied for Question 6.

Conclusion

BUSINESS INTERPRETATION

A LEFT JOIN retains all five customers, including Karan Shah who has no orders. Amit Sharma leads revenue at 3,100, followed by Priya Patil and Rahul Mehta at 3,000 each. The NULL for Karan is useful for identifying inactive customers; COALESCE could be used to present it as zero revenue.

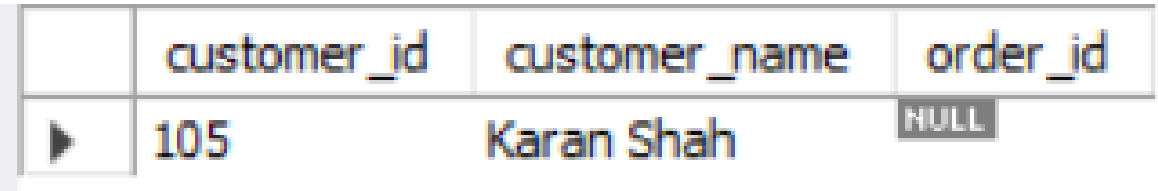
7. Customers Who Have Never Ordered

QUESTION
Customers Who Have Never Ordered

SQL Query

```
SELECT
c.customer_id, c.customer_name, o.order_id
FROM
customers c
LEFT JOIN
orders o ON c.customer_id = o.customer_id
WHERE
o.order_id IS NULL;
```

Output



	customer_id	customer_name	order_id
▶	105	Karan Shah	NULL

Output evidence supplied for Question 7.

Conclusion

BUSINESS INTERPRETATION
Karan Shah is the only customer in the supplied data with no order. This is a clear customer-activation opportunity because the customer exists in the master table but has no transaction history.

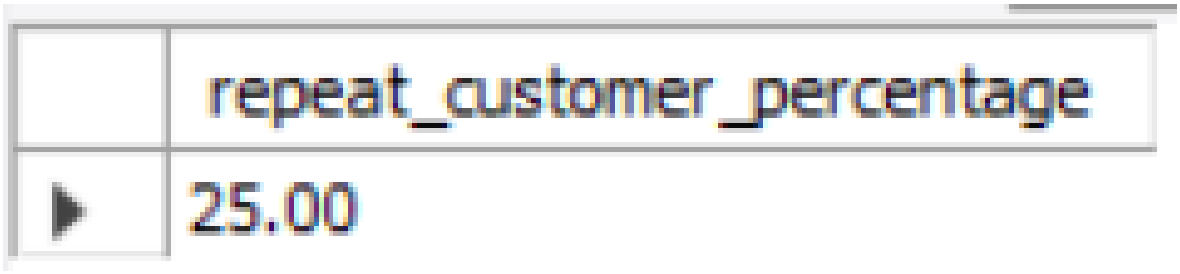
8. What percentage of customers are repeat customers?

QUESTION
What percentage of customers are repeat customers?

SQL Query

```
with repeat_customers as (
SELECT customer_id, COUNT(order_id) AS order_count
FROM orders
GROUP BY customer_id )
SELECT
ROUND(
(SUM(order_count > 1) / COUNT(*)) * 100,
2
) AS repeat_customer_percentage
FROM repeat_customers;
```

Output



Output evidence supplied for Question 8.

Conclusion

BUSINESS INTERPRETATION
The output is 25%. The calculation is based on customers who placed at least one order: 1 of 4 ordering customers placed more than one order. It is not 1 of 5 registered customers; the denominator is the repeat_customer CTE result.

9. Identify the top 3 customers by total revenue.

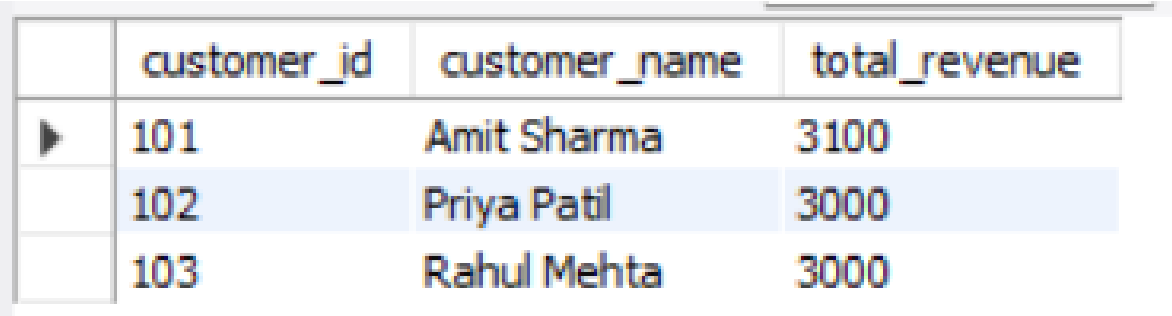
QUESTION

Identify the top 3 customers by total revenue.

SQL Query

```
SELECT
c.customer_id,
c.customer_name,
SUM(o.quantity * o.unit_price) AS total_revenue
FROM
customers c
JOIN
orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id
ORDER BY SUM(o.quantity * o.unit_price) DESC
LIMIT 3;
```

Output



customer_id	customer_name	total_revenue
101	Amit Sharma	3100
102	Priya Patil	3000
103	Rahul Mehta	3000

Output evidence supplied for Question 9.

Conclusion

BUSINESS INTERPRETATION

Amit Sharma is the top customer by gross revenue at 3,100. Priya Patil and Rahul Mehta follow at 3,000 each. The top three customers are tightly clustered, suggesting a relatively balanced high-value group in this small sample.

10. Find customers who have placed more than 1 orders.

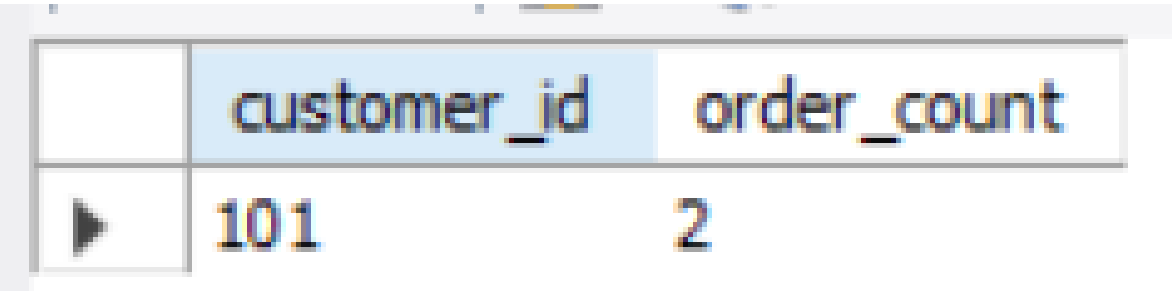
QUESTION

Find customers who have placed more than 1 orders.

SQL Query

```
SELECT
customer_id, COUNT(order_id) AS order_count
FROM
orders
GROUP BY customer_id
HAVING COUNT(order_id) > 1;
```

Output



customer_id	order_count
101	2

Output evidence supplied for Question 10.

Conclusion

BUSINESS INTERPRETATION

Customer 101 has placed two orders, making Amit Sharma the only customer with more than one order. This supports the 25% repeat-customer result among ordering customers.

11. Calculate revenue before and after discount. (groups based on discount then find revenue based on group. Use case)

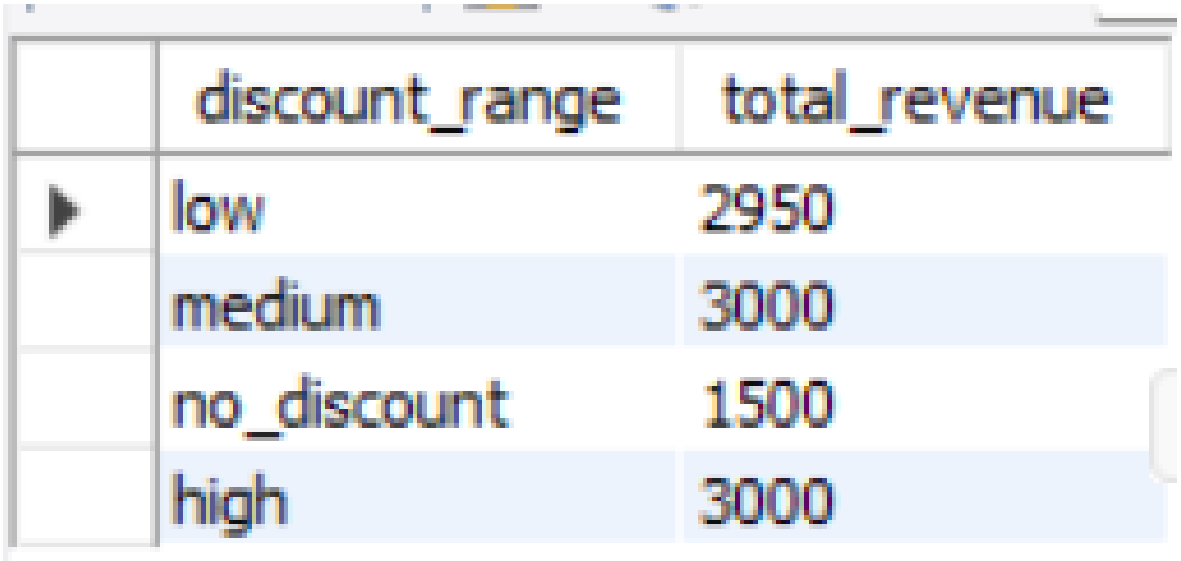
QUESTION

Calculate revenue before and after discount. (groups based on discount then find revenue based on group. Use case)

SQL Query

```
SELECT
CASE
WHEN discount_pct = 0 THEN 'no_discount'
WHEN discount_pct = 5 THEN 'low'
WHEN discount_pct = 10 THEN 'medium'
WHEN discount_pct = 15 THEN 'high'
END AS discount_range,
SUM(quantity * unit_price) AS total_revenue
FROM
orders
GROUP BY CASE
WHEN discount_pct = 0 THEN 'no_discount'
WHEN discount_pct = 5 THEN 'low'
WHEN discount_pct = 10 THEN 'medium'
WHEN discount_pct = 15 THEN 'high'
END;
```

Output



discount_range	total_revenue
low	2950
medium	3000
no_discount	1500
high	3000

Output evidence supplied for Question 11.

Conclusion

BUSINESS INTERPRETATION

The output groups gross revenue by discount bucket: low 2,950; medium 3,000; no discount 1,500; high 3,000. However, the query does not calculate revenue after discount despite the question wording. It only reports gross revenue within each discount group.

Validation Note

CHECK BEFORE PRESENTATION

Q11: The question says before and after discount, but the SQL only aggregates gross revenue by discount bucket.

12. Calculate monthly: Orders, Gross revenue, Discount amount, Net revenue

QUESTION

Calculate monthly: Orders, Gross revenue, Discount amount, Net revenue

SQL Query

```
SELECT
MONTHNAME(order_date) AS month_name,
COUNT(order_id) AS order_counts,
SUM(quantity * unit_price) AS total_revenue,
AVG(discount_pct)
FROM
orders
GROUP BY MONTHNAME(order_date);
```

Output

month_name	order_counts	total_revenue	AVG(discount_pct)
January	2	4600	7.5000
February	3	5850	6.6667

Output evidence supplied for Question 12.

Conclusion

BUSINESS INTERPRETATION

January has 2 orders and 4,600 gross revenue with a 7.50% average discount; February has 3 orders and 5,850 gross revenue with a 6.67% average discount. February has higher order volume and gross revenue. The query does not actually return discount amount or net revenue, despite the question wording.

Validation Note

CHECK BEFORE PRESENTATION

Q12: The question asks for gross revenue, discount amount and net revenue, but the SQL returns gross revenue and average discount percentage.

13. Month-over-Month Revenue Growth

QUESTION

Month-over-Month Revenue Growth

SQL Query

```
WITH monthly_revenue AS
( SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
SUM( quantity * unit_price - quantity * unit_price * discount_pct / 100 ) AS revenue
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
),

revenue_with_previous AS
( SELECT month,
revenue,
LAG(revenue) OVER ( ORDER BY month) AS previous_month_revenue
FROM monthly_revenue
)
SELECT month,
revenue,
previous_month_revenue,
ROUND(
(revenue - previous_month_revenue) * 100.0 / previous_month_revenue,2) AS mom_growth_percentage
FROM revenue_with_previous;
```

Output

	month	revenue	previous_month_revenue	mom_growth_percentage
▶	2024-01	4220.0000	NULL	NULL
	2024-02	5332.5000	4220.0000	26.36

Output evidence supplied for Question 13.

Conclusion

BUSINESS INTERPRETATION

Net revenue increased from 4,220 in January to 5,332.50 in February, producing 26.36% MoM growth. The first month has no prior month, so its growth field is NULL. This is the strongest explicit growth indicator in the supplied analysis.

14. For every category, find the product with the highest revenue.

QUESTION

For every category, find the product with the highest revenue.

SQL Query

```
With city_ranked_customer AS (
SELECT
o.product_category,
c.customer_id,
SUM(o.quantity * o.unit_price) AS total_revenue,
rank () over (partition by city order by SUM(o.quantity * o.unit_price) desc) as revenue_rank
FROM
customers c
JOIN
orders o ON c.customer_id = o.customer_id
group by o.product_category, c.customer_id
)
select * from city_ranked_customer where revenue_rank= 1;
```

Output

	product_category	customer_id	total_revenue	revenue_rank
▶	Fashion	103	3000	1
	Grocery	102	3000	1
	Electronics	101	3100	1

Output evidence supplied for Question 14.

Conclusion

BUSINESS INTERPRETATION

The supplied output shows one customer per category with revenue_rank = 1, not the highest-revenue product per category. The query partitions by city and ranks customer revenue, so it does not answer the stated product-level question. This should be corrected before presentation if the intended question is product performance.

Validation Note

CHECK BEFORE PRESENTATION

Q14: The stated question is product-level, but the SQL/output is customer-level and partitions by city.

15. Which category has the worst average delivery performance?

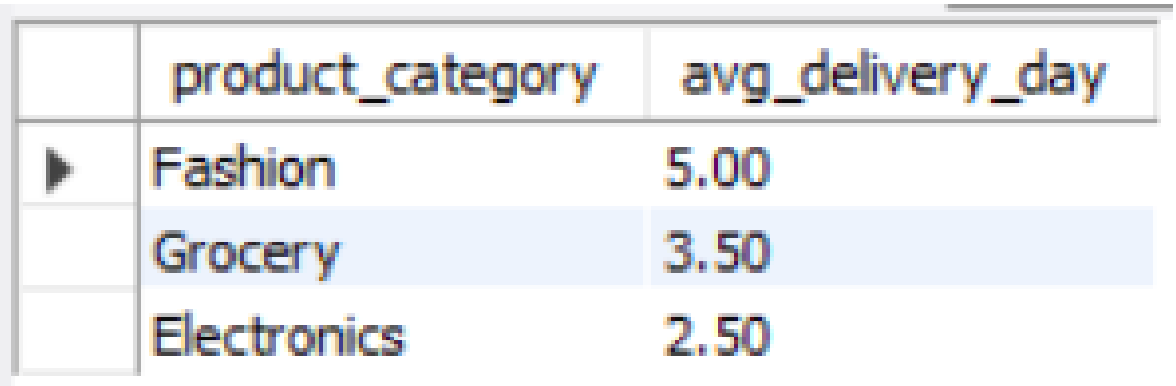
QUESTION

Which category has the worst average delivery performance?

SQL Query

```
SELECT
product_category,
ROUND(Avg(delivery_days),2) as avg_delivery_day
FROM
orders
GROUP BY product_category
ORDER BY avg_delivery_day DESC;
```

Output



product_category	avg_delivery_day
Fashion	5.00
Grocery	3.50
Electronics	2.50

Output evidence supplied for Question 15.

Conclusion

BUSINESS INTERPRETATION

Fashion has the worst average delivery performance at 5.0 days, followed by Grocery at 3.5 days and Electronics at 2.5 days. Fashion is therefore the priority category for delivery-performance investigation.

16. Find the top 3 customers by revenue within every city.

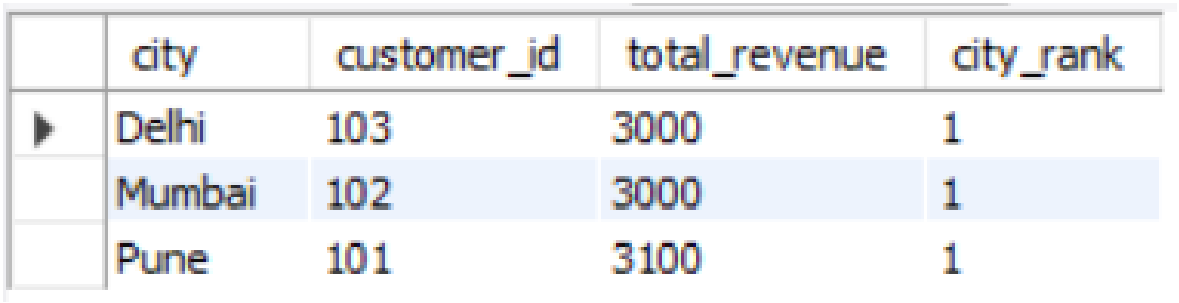
QUESTION

Find the top 3 customers by revenue within every city.

SQL Query

```
With city_ranked_customer AS (
SELECT
c.city,
c.customer_id,
SUM(o.quantity * o.unit_price) AS total_revenue,
rank () over (partition by city order by SUM(o.quantity * o.unit_price) desc) as city_rank
FROM
customers c
JOIN
orders o ON c.customer_id = o.customer_id
group by c.city, c.customer_id
)
select * from city_ranked_customer where city_rank= 1
```

Output



	city	customer_id	total_revenue	city_rank
▶	Delhi	103	3000	1
	Mumbai	102	3000	1
	Pune	101	3100	1

Output evidence supplied for Question 16.

Conclusion

BUSINESS INTERPRETATION

The output shows rank 1 customers for Delhi, Mumbai and Pune. The query filters city_rank = 1, so it returns the top customer rather than the requested top three. The intended top-three version should filter city_rank <= 3. In the current five-customer sample, only one active customer appears in each represented city.

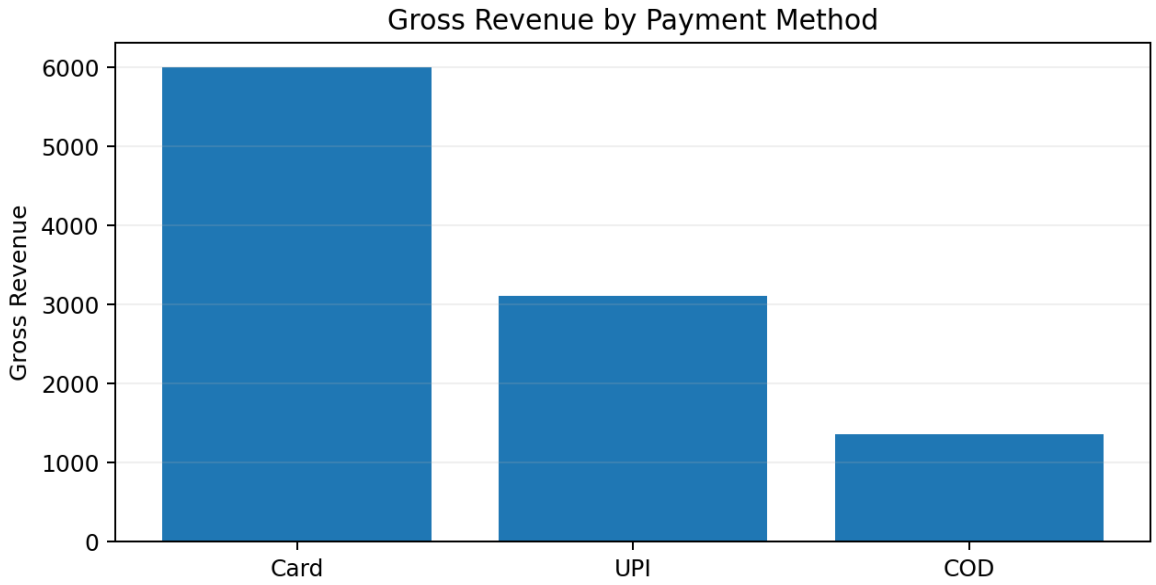
Validation Note

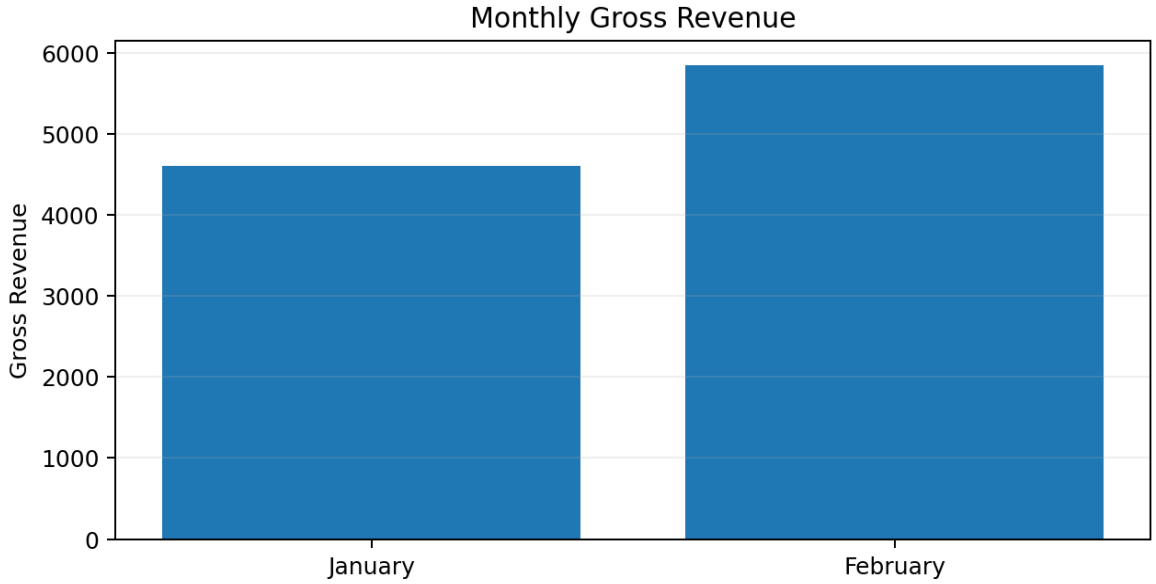
CHECK BEFORE PRESENTATION

Q16: The stated question asks for top 3 per city, but the SQL filters rank = 1, returning only the top 1 per city.

3. Key Business Insights

Area	Finding
Revenue	Gross revenue is 10,450. Grocery leads with 4,350, ahead of Electronics (3,100) and Fashion (3,000).
Payments	Card is the strongest payment method by gross revenue at 6,000.
Customers	Amit Sharma is the highest-revenue customer at 3,100; Karan Shah has no orders.
Retention	25% of ordering customers are repeat customers under the supplied CTE definition.
Growth	Net revenue rises from 4,220 in January to 5,332.50 in February, a 26.36% MoM increase.
Operations	Fashion has the highest average delivery time at 5.0 days, indicating the clearest operational issue.





4. Query Validation & Presentation Notes

The report intentionally preserves the supplied SQL and screenshots. The following items are flagged rather than silently corrected.

REVIEW
Q1: The question asks for total orders, but the query counts rows from customers. The displayed result is coincidentally 5 because both tables contain five rows.

REVIEW
Q3: The screenshot repeats Q2's gross-revenue result, while the SQL calculates an average. Expected AOV from the supplied data/query is 2,090.

REVIEW
Q11: The question says before and after discount, but the SQL only aggregates gross revenue by discount bucket.

REVIEW
Q12: The question asks for gross revenue, discount amount and net revenue, but the SQL returns gross revenue and average discount percentage.

REVIEW
Q14: The stated question is product-level, but the SQL/output is customer-level and partitions by city.

REVIEW
Q16: The stated question asks for top 3 per city, but the SQL filters rank = 1, returning only the top 1 per city.

Recommended next steps

• Correct Q1 to count orders.order_id for a reliable order KPI.

• Refresh Q3 output so the displayed result matches the AVG query.

• Extend Q11 to calculate both gross revenue and net revenue/discount amount by discount bucket.

• Extend Q12 to explicitly calculate discount amount and net revenue by month.

• Rewrite Q14 at product level if the business question is highest-revenue product by category.

• Change Q16 filter to city_rank <= 3 if the intended output is the top three customers in every city.

• Add order status filtering if cancelled/returned orders are introduced into the dataset.

• Scale the analysis with more months and more customers before making strategic decisions.

5. Final Conclusion

The SQL analysis demonstrates a solid progression from basic transaction KPIs to customer analysis, categorical revenue analysis, discount segmentation, monthly trends, window functions and operational performance. The strongest business story in the supplied sample is: Grocery leads revenue, Card dominates payment revenue, Amit Sharma is the top customer, one registered customer is inactive, repeat-customer penetration is 25% among ordering customers, February net revenue grows 26.36% MoM, and Fashion has the slowest delivery performance.

Presentation takeaway

Use the corrected/validated versions of Q1, Q3, Q11, Q12, Q14 and Q16 before presenting this project to an interviewer or business audience. The underlying analysis concepts are strong; aligning each question, query and output will make the project much more credible.

Source basis: E-Commerce Full Analysis.sql + E commerce sales sql outputs.docx